LangChain

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

1. Introduction to LangChain

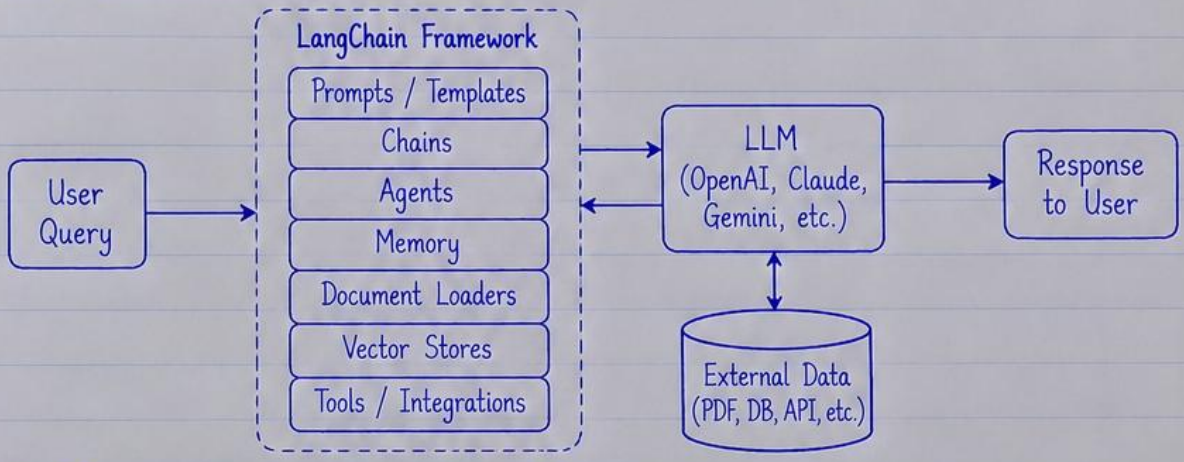
1 What is LangChain ?

- LangChain is an open-source framework designed to build applications with Large Language Models (LLMs).
- It provides tools, abstractions and modules to connect LLMs with external data, tools and real-world actions.
- It simplifies the development of complex LLM applications like chatbots, agents, RAG systems, document analysis, etc.
- LangChain is language-agnostic but most popular with Python.

2 Why LangChain ?

- Provides high-level abstractions for LLM apps.
- Easy integration with multiple LLM providers (OpenAI, Anthropic, Google, etc).
- Built-in support for document loaders, text splitters, vector stores and tools.
- Helps in building production-ready applications.
- Modular and extensible architecture.
- Large community and active development.
- Useful for building RAG, agents, chatbots, and multi-step workflows.

3 High Level Architecture



4 Core Concepts in LangChain

- Models – LLMs and chat models.
- Prompts – Templates to create structured inputs.
- Chains – Sequence of operations using LLMs.
- Agents – Use tools to perform actions.
- Memory – Maintain conversation history.
- Document Loaders – Load data from various sources.
- Text Splitters – Split large documents.
- Vector Stores – Store and search embeddings.
- Retrievers – Fetch relevant documents.
- Tools – Integrate with external APIs, DBs, etc.
- Callbacks – Track and monitor token usage.
- Output Parsers – Structure the LLM output.

5 LangChain vs Direct LLM API Calls

Aspect	Direct LLM API Calls	LangChain
Ease of Use	Manual implementation	High-level abstractions
Prompt Management	Hard-coded	Prompt templates and chains
Document Handling	Custom implementation	Built-in document loaders and splitters
Vector Store Integration	Manual integration	Direct support (FAISS, Chroma, etc.)
Agents and Tools	Complex to implement	Built-in agents and tool integration
Memory	Manual handling	Built-in memory modules
Production Ready	More effort	Ready-to-use components

Rahul Yadav

3. Setting Up LangChain (Environment and Installation)

1 Prerequisites

- Python 3.8 or higher
- Basic knowledge of Python
- Familiarity with LLM concepts (optional)
- API keys for LLM providers (e.g., OpenAI)
- Virtual environment (recommended)
- Code editor (VS Code, PyCharm, etc.)

2 Create Virtual Environment

```
# Create virtual environment
python -m venv langchain_env

# Activate (Windows)
langchain_env\Scripts\activate

# Activate (Mac/Linux)
source langchain_env/bin/activate
```

3 Install LangChain and Required Packages

```
# Install LangChain (latest version)
pip install langchain

# Install community integrations
pip install langchain-community

# Install OpenAI support
pip install langchain-openai

# Other useful packages
pip install python-dotenv
pip install faiss-cpu
pip install chromadb
pip install tiktoken
pip install pypdf
```

4 Set Up API Keys

- Create an account at OpenAI (or other provider).
- Generate an API key.
- Keep the API key secure (do not hardcode).
- Use environment variables for security.
- Create a .env file in your project root.

.env file

```
OPENAI_API_KEY=your_api_key_here
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=your_langchain_key
```

5 Load Environment Variables in Code

```
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")
print(api_key)
```

6 Verify Installation

```
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-3.5-turbo")
response = llm.invoke("Hello! LangChain is working.")
print(response.content)
```

7 Project Structure (Simple)

```
langchain-project/
|-- .env
|-- app.py
|-- requirements.txt
|-- data/
|   `-- sample.pdf
|-- notebooks/
|-- README.md
```

8 requirements.txt Example

```
langchain
langchain-community
langchain-openai
python-dotenv
faiss-cpu
chromadb
pypdf
tiktoken
```

9 Common Issues and Solutions

Issue	Solution
ModuleNotFoundError	Install the missing package using pip
Invalid API Key	Check API key and .env file
Version conflicts	Use compatible versions or create new venv
Rate limit error	Use smaller requests or different model
Import errors	Use correct import path (langchain_community, etc.)
Environment not activated	Activate virtual environment before running code

Rahul Yadav

4. LLM Models and Integrations

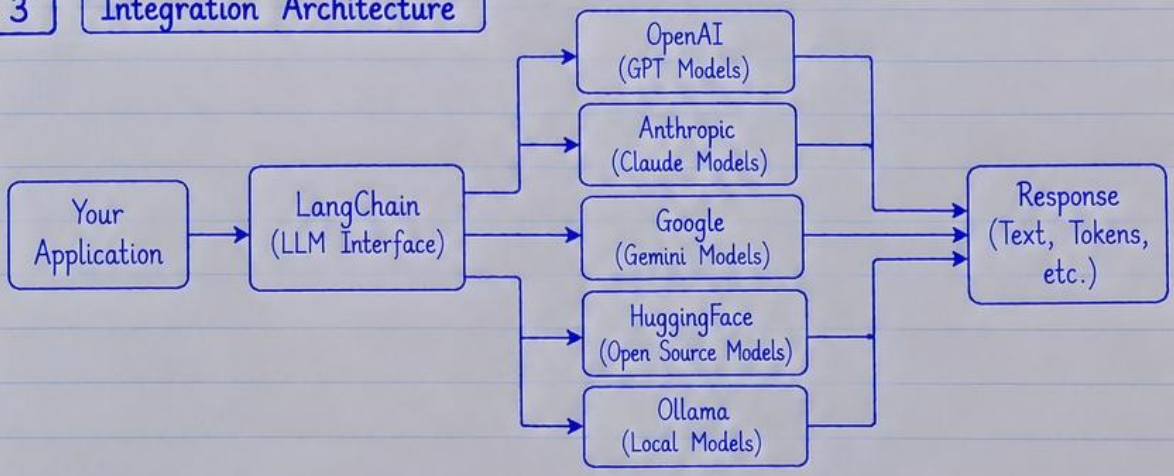
1 What are LLMs ?

- Large Language Models (LLMs) are AI models trained on massive text data.
- They can understand, generate and manipulate human language.
- Examples: OpenAI (GPT), Anthropic (Claude), Google (Gemini), Meta (Llama) etc.
- LangChain provides easy integration with multiple LLM providers through a unified interface.

2 Supported LLM Providers in LangChain

- OpenAI (GPT-3.5, GPT-4, GPT-4o)
- Anthropic (Claude)
- Google (Gemini)
- HuggingFace (Open source models like Llama)
- Ollama (Local LLMs)
- Cohere
- Azure OpenAI
- and many more via community integrations

3 Integration Architecture



4 OpenAI Integration Example

```
from langchain_openai import ChatOpenAI

# Create LLM instance
llm = ChatOpenAI(
    model="gpt-3.5-turbo",
    temperature=0.7,
    api_key="your_api_key"
)

# Simple invocation
response = llm.invoke("Explain LangChain in simple words.")
print(response.content)
```

5 Anthropic (Claude) Integration Example

```
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(
    model="claude-3-sonnet-20240229",
    temperature=0.7,
    api_key="your_api_key"
)

response = llm.invoke("What is LangChain?")
print(response.content)
```

6 Using Local LLM with Ollama

```
from langchain_community.llms import Ollama

llm = Ollama(
    model="llama3",
    base_url="http://localhost:11434"
)

response = llm.invoke("Tell me a joke.")
print(response)
```

7 Key Parameters

Parameter	Description
model	Specify the LLM model name
temperature	Controls randomness (0.0 - 1.0)
api_key	API key for the provider
base_url	Required for local models (Ollama)
max_tokens	Maximum number of tokens in response
top_p	Nucleus sampling parameter
stop	List of stop sequences

Rahul Yadav

2. LangChain Architecture and Core Components

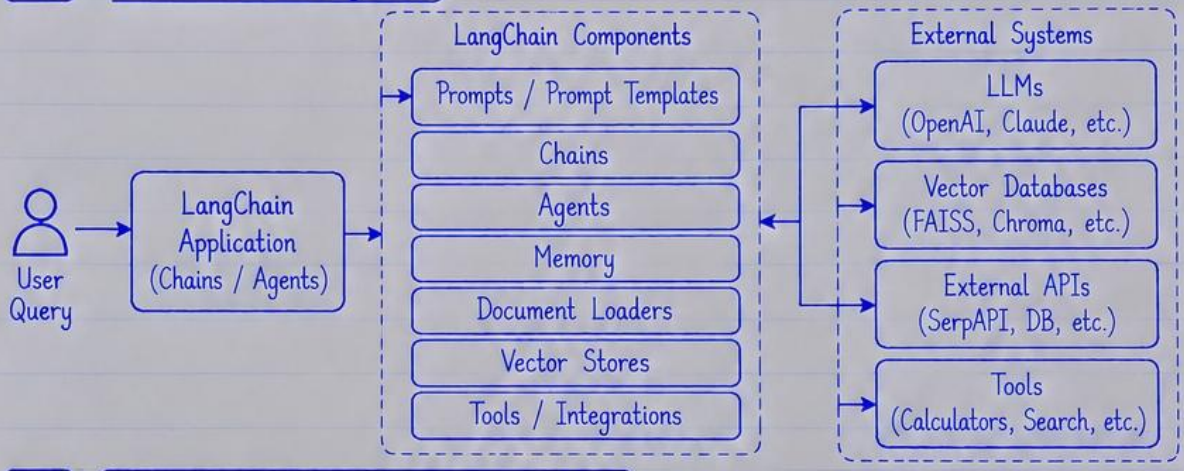
1 LangChain Architecture Overview

- LangChain provides a modular architecture to build LLM applications.
- It acts as a bridge between LLMs, external data sources, tools and application logic.
- It provides standard interfaces and abstractions to work with different LLM providers and components.
- We can mix and match components as per our use case (RAG, Agents, etc.).

2 High Level Architecture

- User provides a query to the application.
- LangChain processes the query using chains, agents or retrievers.
- It interacts with LLMs, external data sources, tools and vector stores.
- LLM generates a response using provided context and tools.
- LangChain returns the final response to the user.

3 Architecture Diagram



4 Core Components and Their Purpose

Component	Purpose / Description
Models	Interface to different LLMs (OpenAI, Anthropic, etc.).
Prompts	Manage and create prompts, prompt templates.
Chains	Combine multiple steps to create a workflow.
Agents	Use tools to take actions and solve complex tasks.
Memory	Maintain conversation history and context.
Document Loaders	Load data from various sources (PDF, Web, DB, etc.).
Text Splitters	Split large documents into smaller chunks.
Vector Stores	Store and search embeddings for similarity search.
Retrievers	Fetch relevant documents from vector store.
Tools	Integrate external tools and APIs.
Callbacks	Track and monitor the execution (logs, token usage, etc.).

5 Simple Example - Using Core Components

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain.chains import LLMChain

# 1. Define LLM
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

# 2. Define prompt template
template = PromptTemplate(
    input_variables=["topic"],
    template="Explain {topic} in simple words."
)

# 3. Create chain
chain = LLMChain(llm=llm, prompt=template)

# 4. Run chain
response = chain.invoke({"topic": "LangChain"})
print(response)
```

- We create an LLM instance (OpenAI).
- Define a prompt template with a variable.
- Create an LLMChain with the model and prompt.
- Invoke the chain with input to get the response.
- This is a simple example. We can also use memory, tools, retrievers, etc., for advanced applications.

Rahul Yadav

5. Prompts and Prompt Templates in LangChain

1 What are Prompts ?

- A prompt is the input instruction given to the LLM.
- It guides the model to generate desired output.
- Good prompts lead to better, accurate and consistent responses.
- LangChain provides prompt templates to create dynamic and reusable prompts.

2 Why Use Prompt Templates ?

- Reusable and maintainable prompts.
- Insert dynamic variables (e.g., user input).
- Separate prompt logic from code.
- Easy to manage complex prompts.
- Support for different prompt types (chat, system, human, etc.).
- Helps in building structured and consistent LLM applications.

3 Types of Prompts in LangChain

Type	Description	Use Case
PromptTemplate	Simple text prompt with variables	Basic Q&A, text generation
ChatPromptTemplate	Chat-based prompt with system, human, AI messages	Chat models (ChatOpenAI, ChatAnthropic, etc.)
MessagesPlaceholder	Placeholder for dynamic message history	Chat with conversation history
FewShotPromptTemplate	Use examples in the prompt	Few-shot learning
PipelinePromptTemplate	Combine multiple templates	Complex multi-step prompts

4 PromptTemplate Example

```
from langchain_core.prompts import PromptTemplate

template = PromptTemplate(
    input_variables=["topic"],
    template="Explain {topic} in simple words."
)

prompt = template.format(topic="LangChain")
print(prompt)
# Output: Explain LangChain in simple words.
```

5 ChatPromptTemplate Example

```
from langchain_core.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful AI assistant."),
    ("human", "Explain {topic} in simple words.")
])

prompt = template.format_messages(topic="LangChain")

# This creates formatted chat messages
# to be used with ChatOpenAI or other chat models.
```

6 Few Shot Prompt Template Example

```
from langchain_core.prompts import FewShotPromptTemplate,
PromptTemplate

examples = [
  {"input": "Python", "output": "A programming language."},
  {"input": "LangChain", "output": "A framework to build
    LLM applications."}
]

example_prompt = PromptTemplate(
    input_variables=["input", "output"],
    template="Input: {input}\nOutput: {output}"
)

few_shot = FewShotPromptTemplate(
    examples=examples,
    example_prompt=example_prompt,
    prefix="Answer the question:",
    suffix="Input: {question}\nOutput:",
    input_variables=["question"]
)
```

7 Prompt Engineering Tips

- Be clear and specific in instructions.
- Provide examples (few-shot) when needed.
- Use structured format (JSON, lists, etc.).
- Set role using system prompts.
- Break complex tasks into smaller steps.
- Use constraints (e.g., length, tone, format).
- Experiment with different phrasings.

8 Example: Using Prompt with LLM

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)
template = ChatPromptTemplate.from_messages([
    ("human", "Explain {topic} in simple words.")
])

chain = template | llm
response = chain.invoke({"topic": "LangChain"})
print(response.content)
```

Rahul Yadav

6. Chains in LangChain

1 What are Chains ?

- A Chain is a sequence of steps (components) that are executed in a logical order.
- It simplifies complex workflows by combining multiple components.
- Chains can include LLMs, prompts, tools, retrievers, memory, etc.
- LangChain provides different types of chains for various use cases.

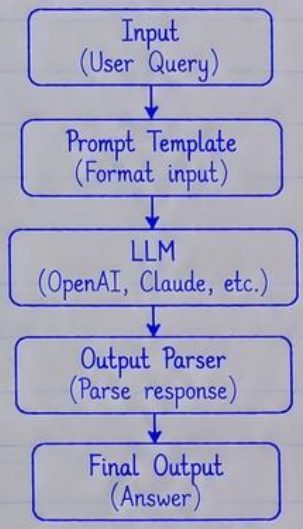
2 Why Use Chains ?

- Helps in building multi-step workflows.
- Easy to combine multiple components.
- Reduces boilerplate code.
- Supports input/output parsing.
- Useful for RAG, agents, document processing, API integrations, etc.
- Provides a standard interface for complex LLM applications.

3 Types of Chains in LangChain

Chain Type	Description
LLMChain	Basic chain with LLM and prompt.
SequentialChain	Run multiple chains in sequence.
SimpleSequentialChain	Sequential chain with simple I/O.
RouterChain	Route input to different chains.
RetrievalQA Chain	Combine retriever with LLM for RAG.
ConversationChain	Maintain conversation (with memory).
MapReduceChain	Process large documents in parallel.
TransformChain	Custom transformation logic.

4 Basic Chain Architecture



5 LLMChain Example

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain.chains import LLMChain

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

prompt = PromptTemplate(
    input_variables=["topic"],
    template="Explain {topic} in simple words."
)

chain = LLMChain(llm=llm, prompt=prompt)

response = chain.invoke({"topic": "Microservices"})
print(response["text"])
```

6 SequentialChain Example

```
from langchain.chains import SequentialChain
from langchain.chains import LLMChain
chain1 = LLMChain(llm=llm, prompt=PromptTemplate(
    input_variables=["topic"],
    template="Give a short summary of {topic)."
)
chain2 = LLMChain(llm=llm, prompt=PromptTemplate(
    input_variables=["summary"],
    template="List key points from this summary:
    {summary}"
)
sequential_chain = SequentialChain(
    chains=[chain1, chain2],
    input_variables=["topic"],
    output_variables=["summary", "points"]
)
result = sequential_chain.invoke({"topic": "LangChain"})
print(result)
```

7 RetrievalQA Chain (RAG) Example

```
from langchain_openai import ChatOpenAI
from langchain.chains import RetrievalQA
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import OpenAIEmbeddings

llm = ChatOpenAI(model="gpt-3.5-turbo")
embedding = OpenAIEmbeddings()
vectorstore = FAISS.load_local("./faiss_index", embedding,
              allow_dangerous_deserialization=True)
retriever = vectorstore.as_retriever()

chain = RetrievalQA.from_chain_type(llm=llm,
            retriever=retriever)

response = chain.invoke({"query": "What is LangChain?"})
print(response["result"])
```

8 Key Takeaways

- Chains help in creating structured and reusable workflows.
- Choose the right chain based on your use case.
- Use RetrievalQA chain for RAG applications.
- Sequential chains are useful for multi-step tasks.
- We can also create custom chains when needed.
- Chains can be combined with memory, tools, and output parsers.

Rahul Yadav

7. Agents in LangChain

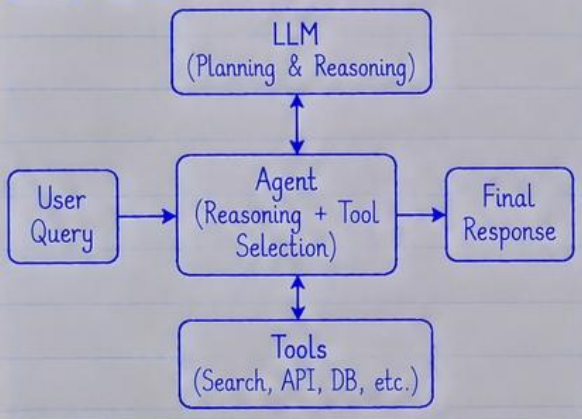
1 What are Agents ?

- Agents are LLMs that can take actions using tools to achieve a goal.
- They can decide which tool to use, when to use it and what input to pass.
- Agents follow a reasoning process and can work in multi-step workflows.
- LangChain provides a standard interface to build agents with different types of tools and strategies.

2 How Agents Work ?

1. Receive user input (goal / question).
2. Understand the task and plan the next step.
3. Select the appropriate tool from available tools.
4. Execute the tool with required input.
5. Observe the result and decide next action.
6. Repeat the process until final answer is ready.

3 Agent Architecture



4 Types of Agents in LangChain

Agent Type	Description
Zero-shot Agent	Uses LLM to decide tool usage without examples.
ReAct Agent	Uses reasoning (Thought → Action → Observation) pattern.
Structured Chat Agent	Works with structured chat format.
OpenAI Functions Agent	Uses OpenAI function calling.
Tool Calling Agent	Uses tool calling models (e.g., GPT-4).
Self-Ask with Search	Breaks complex questions into simpler sub-questions.

5 Creating a Simple Agent with Tools

```
from langchain_openai import ChatOpenAI
from langchain.agents import initialize_agent, AgentType
from langchain.tools import DuckDuckGoSearchRun

# Initialize LLM
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)
# Define tools
search = DuckDuckGoSearchRun()
tools = [search]

# Create agent
agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True
)

# Run agent
response = agent.invoke("What is LangChain?")
print(response["output"])
```

6 Example: Agent with Multiple Tools

```
from langchain_openai import ChatOpenAI
from langchain.agents import initialize_agent, AgentType
from langchain.tools import DuckDuckGoSearchRun, PythonREPLTool

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

search = DuckDuckGoSearchRun()
python_tool = PythonREPLTool()
tools = [search, python_tool]

agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.OPENAI_FUNCTIONS,
    verbose=True
)

response = agent.invoke("Search latest LangChain news
          and calculate 5 * 12 using Python.")
print(response["output"])
```

7 Use Cases of Agents

- Answer complex questions using multiple tools.
- Search the internet for latest information.
- Query internal databases.
- Interact with APIs (e.g., weather, finance, etc.).
- Perform calculations using Python.
- Build research assistants, travel planners, report generators. Automate multi-step tasks.
- Combine multiple tools for real-world applications.

8 Key Takeaways

- Agents can use tools to perform actions.
- Choose the right agent type for your use case.
- Provide clear tool descriptions.
- Use ReAct or Function calling for better results.
- Agents are useful for complex, multi-step tasks.
- Always validate and restrict tool access for security.
- Experiment with different tools and prompts.

Rahul Yadav

8. Memory in LangChain

1 What is Memory ?

- Memory allows LLM to remember previous interactions.
- It helps in maintaining conversation context across multiple turns.
- Without memory, each request is treated independently and the model forgets previous messages.
- LangChain provides different types of memory based on the use case.

2 Why Use Memory ?

- Maintain conversation history.
- Provide contextual responses.
- Useful for chatbots, virtual assistants and agents.
- Improves user experience.
- Supports short-term and long-term memory.
- Helps in building more human-like conversations.

3 Types of Memory in LangChain

Memory Type	Description	Use Case
ConversationBufferMemory	Stores full conversation history in memory.	Simple chat applications.
ConversationBufferWindowMemory	Stores last k messages.	Limit memory size.
ConversationSummaryMemory	Summarizes old messages.	Long conversations.
EntityMemory	Remembers important entities (e.g., names, facts).	Personalized assistants.
VectorStoreRetrieverMemory	Stores conversation history in vector store.	Long-term memory with retrieval.

4 ConversationBufferMemory Example

```
from langchain_openai import ChatOpenAI
from langchain_memory import ConversationBufferMemory
from langchain.chains import ConversationChain

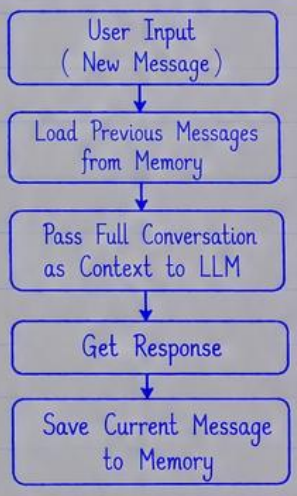
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

memory = ConverstationBufferMemory()

conversation = ConverstationChain(
    llm=llm,
    memory=memory,
    verbose=True
)

# First message
response1 = conversation.predict(input="Hi, my name is Rahul.")
print(response1)
# Second message (model remembers previous message)
response2 = conversation.predict(input="What is my name?")
print(response2)
```

5 How ConversationBufferMemory Works



6 ConversationBufferWindowMemory Example

```
from langchain_memory import ConversationBufferWindowMemory
from langchain.chains import ConversationChain

memory = ConversationBufferWindowMemory(k=3)

conversation = ConverstationChain(
    llm=llm,
    memory=memory,
    verbose=True
)

# This will keep only last 3 messages in memory
```

7 ConversationSummaryMemory Example

```
from langchain_memory import ConverstationSummaryMemory
memory = ConversationSummaryMemory(llm=llm)
conversation = ConverstationChain(
    llm=llm,
    memory=memory,
    verbose=True
)
```

8 Key Takeaways

- Memory helps in maintaining context in conversations.
- Choose the right memory type based on your use case.
- Use summary memory for long conversations.
- Vector store memory is useful for long-term storage.
- Memory can also be used with agents and chains.

Rahul Yadav

9. Tools in LangChain

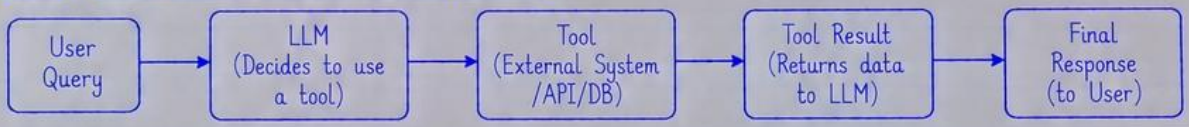
1 What are Tools ?

- Tools allow LLMs to interact with external systems (APIs, databases, search engines, etc.).
- LLMs can call tools to get real-time information, perform actions, or execute code.
- LangChain provides a standard interface to create, integrate, and manage tools.
- Tools are essential for building agents and real-world applications.

2 Why Use Tools ?

- Access real-time and external information.
- Perform actions like search, calculation, API calls.
- Extend LLM capabilities beyond training data.
- Useful for building RAG, agents, and automation.
- Easily create custom tools.
- Integrate with popular services (Google Search, Wikipedia, Python REPL, databases, etc.).

3 How Tools Work in LangChain



4 Types of Tools in LangChain

Tool Type	Description	Use Case
Built-in Tools	Pre-built tools provided by LangChain	Search, Wikipedia, Python REPL, etc.
API Tools	Call external APIs	Weather, Finance, Custom APIs
Database Tools	Interact with databases	Query MySQL, PostgreSQL, etc.
Custom Tools	Create your own tool using function	Business logic, internal services
Toolkits	Collection of related tools	SerpAPI toolkit, SQL toolkit, etc.

5 Popular Built-in Tools

- SerpAPIWrapper - Google Search
- WikipediaQueryRun - Wikipedia search
- PythonREPLTool - Execute Python code
- RequestsToolkit - Make HTTP requests
- SQLDatabaseToolkit - Work with SQL databases
- ArxivQueryRun - Search research papers
- DuckDuckGoSearchRun - Web search (open source)
- OpenWeatherMap - Weather information
- ... and many more via LangChain community

6 Creating a Custom Tool

- Define a function with input and output.
- Use the **@tool** decorator or BaseTool.
- Provide a clear description for the LLM.
- Can be used with agents or chains.

7 Example 1: Using SerpAPI Google Search Tool

```
from langchain.agents import initialize_agent, AgentType
from langchain_community.utilities import SerpAPIWrapper
from langchain.tools import Tool
from langchain_openai import ChatOpenAI

# Initialize LLM
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

# Create search tool
search = SerpAPIWrapper()
tools = [Tool(
        name="Google Search",
        func=search.run,
        description="Search the internet for latest information")
)]
# Initialize agent
agent = initialize_agent(tools, llm,
        agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
        verbose=True)

response = agent.invoke({"input": "Who won the IPL 2024?"})
print(response["output"])
```

8 Example 2: Custom Tool (Calculator)

```
from langchain.tools import tool
from langchain.agents import initialize_agent, AgentType
from langchain_openai import ChatOpenAI

@tool
def calculator(expression: str) -> str:
      """Useful for mathematical calculations.
      Input should be a valid Python expression."""
      try:
         return str(eval(expression))
      except Exception as e:
         return f"Error: {str(e)}"
)
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)
tools = [calculator]

agent = initialize_agent(tools, llm,
       agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
       verbose=True)

response = agent.invoke({"input": "What is 25 * 4 + 10?"})
print(response["output"])
```

9 Using Multiple Tools with an Agent

```
from langchain_community.utilities import SerpAPIWrapper
from langchain.tools import Tool
search_tool = Tool(name="Search", func=SerpAPIWrapper().run,
  description="Search the internet")
calc_tool = calculator  # from previous example
tools = [search_tool, calc_tool]
agent = initialize_agent(tools, llm,
        agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
        verbose=True)
```

10 Key Takeaways

- Tools extend LLM capabilities to real-world applications.
- Use built-in tools for common tasks.
- Create custom tools for business-specific needs.
- Tools are the building blocks for powerful agents.
- Provide clear descriptions for better tool selection.
- Combine multiple tools for complex workflows.
- Experiment with different tools and use cases.

Rahul Yadav

10. Agents in LangChain (Advanced)

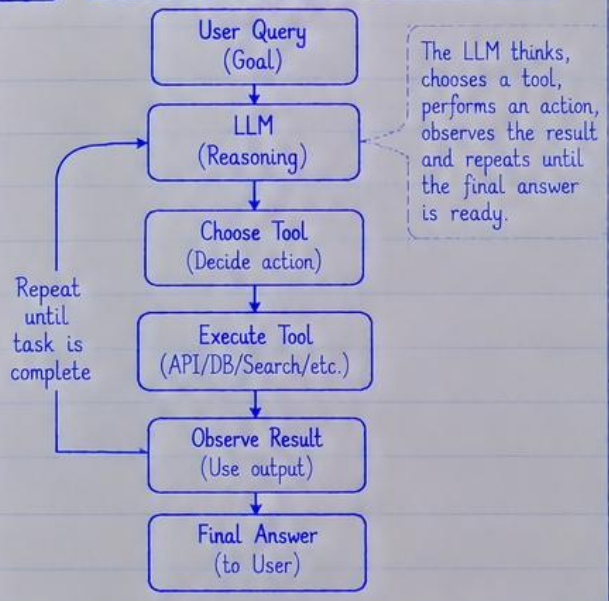
1 What are Agents ?

- Agents are LLMs that can decide which tools to use, when to use them, and in what order to achieve a goal.
- They can break complex tasks into multiple steps, use tools, and reason about the next action.
- Agents follow a reasoning loop called "Plan → Action → Observation".
- LangChain provides different agent types and easy way to create custom agents.

2 Why Use Agents ?

- Handle complex, multi-step tasks.
- Automatically choose the right tool based on the user query.
- Can work with real-time data (APIs, DB, search, etc.).
- Useful for building chatbots, research assistants, report generators, and automation systems.
- Reduce manual effort by letting LLM decide the workflow.

3 How Agents Work ? (ReAct Pattern)



4 Types of Agents in LangChain

Agent Type	Description
ZeroShotAgent	Decides tool usage without examples.
ReActAgent	Uses reasoning (Thought → Action → Observation) to decide next step.
StructuredChatAgent	Uses structured chat format with tools.
OpenAIFunctionsAgent	Uses OpenAI function calling.
ConversationalAgent	Maintains conversation history.
SelfAskWithSearchAgent	Breaks complex questions into simple sub-questions.
Custom Agent	Create your own agent with custom logic and tools.

5 Common Tools Used with Agents

- SerpAPI/Search tools (Google Search)
- Python REPL tool (run Python code)
- Requests tool (HTTP API calls)
- SQL Database tool (query database)
- Wikipedia tool (get general knowledge)
- Custom tools (business logic, internal APIs)

6 Example 1: ReAct Agent with Search Tool

```
from langchain_openai import ChatOpenAI
from langchain.agents import initialize_agent, AgentType
from langchain.tools import SerpAPIWrapper

# Initialize LLM
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

# Create search tool
search = SerpAPIWrapper()
tools = [search]

# Create ReAct agent
agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.REACT_DOCSTORE,
    verbose=True
)

response = agent.invoke({"input": "What is the latest news about electric vehicles in India?"})
print(response["output"])
```

7 Example 2: Agent with Multiple Tools

```
from langchain_openai import ChatOpenAI
from langchain.agents import initialize_agent, AgentType
from langchain.tools import SerpAPIWrapper, PythonREPLTool

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

search = SerpAPIWrapper()
python_tool = PythonREPLTool()
tools = [
    Tool(name="Search", func=search.run,
      description="Search the internet for information"),
    Tool(name="PythonREPL", func=python_tool.run,
      description="Run Python code for calculation or analysis")
]
agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True
)
response = agent.invoke({"input": "Search the population of Delhi and calculate 5% of that number using Python."})
print(response["output"])
```

8 Example 3: Custom Tool with Agent

```
from langchain.tools import tool
@tool
def get_employee_details(emp_id: str) -> str:
    """Get employee details from internal system."""
    # Your business logic here
    return f"Employee {emp_id} - Name: Rahul, Dept: IT"

# Use this tool with an agent
# tools = [get_employee_details]
# agent = initialize_agent(tools, llm, agent=..., verbose=True)
```

9 Key Takeaways

- Agents can use multiple tools to solve complex tasks.
- Choose the right agent type based on your use case.
- Provide clear tool descriptions for better results.
- Agents follow a reasoning loop (Plan → Action → Observation).
- Combine search, database, code execution, and custom tools.
- Useful for building powerful AI assistants and automation workflows.

Rahul Yadav

11. LangChain with LLM Providers

1 What are LLM Providers ?

- LLM providers offer powerful language models through APIs or local deployment.
- LangChain supports multiple LLM providers with a unified interface.
- You can easily switch between providers without changing your application logic.
- Popular providers include OpenAI, Anthropic, Google, Cohere, Hugging Face, and local models like Llama (via Ollama).

2 Why Use Multiple LLM Providers ?

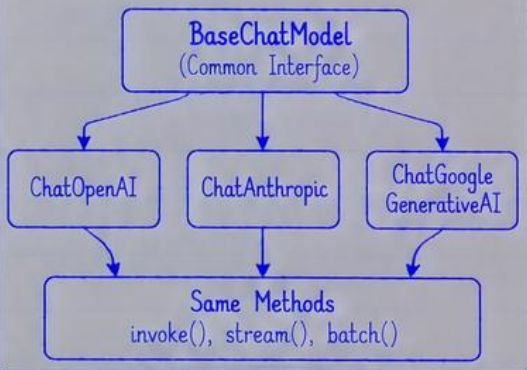
- Choose the best model for your use case.
- Compare performance, cost, and speed.
- Switch providers easily using LangChain.
- Use local models for privacy and cost saving.
- Fallback to another provider if one fails.
- Experiment with different models for better results.
- Supports both cloud-based and local LLMs.

3 Popular LLM Providers Supported by LangChain

Provider	Model Examples	How to Use
OpenAI	GPT-4, GPT-3.5	ChatOpenAI
Anthropic	Claude 3, Claude 2	ChatAnthropic
Google	Gemini Pro, Gemini Flash	ChatGoogleGenerativeAI
Cohere	Command R, Command R+	ChatCohere
Hugging Face	Open source models	HuggingFacePipeline
Ollama (Local)	Llama 3, Mistral, Phi	Ollama
Azure OpenAI	GPT-4 (Azure)	AzureChatOpenAI
AWS Bedrock	Claude, Llama, etc.	BedrockChat

4 Unified Interface in LangChain

```
from langchain_core.language_models import BaseChatModel
```



5 Example 1: OpenAI (GPT-4)

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4", temperature=0.7)

response = llm.invoke("Explain microservices
          in simple words.")
print(response.content)
```

6 Example 2: Anthropic (Claude)

```
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(model="claude-3-sonnet-20240229",
                    temperature=0.7)

response = llm.invoke("What is LangChain?")
print(response.content)
```

7 Example 3: Google Gemini

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleSenerativeAI(
     model="gemini-pro",
     temperature=0.7)
)
response = llm.invoke("List key features of LangChain.")
print(response.content)
```

8 Example 4: Local LLM with Ollama

```
from langchain_community.llms import Ollama

llm = Ollama(model="llama3")

response = llm.invoke("Explain RAG in simple words.")
print(response)
```

9 Example 5: Hugging Face Model

```
from langchain_huggingface import HuggingFacePipeline
from transformers import pipeline

pipe = pipeline("text-generation",
          model="mistralai/Mistral-7B-Instruct-v0.2",
          max_new_tokens=200)

llm = HuggingFacePipeline(pipeline=pipe)

response = llm.invoke("What is Docker?")
print(response)
```

10 Key Takeaways

- LangChain supports multiple LLM providers.
- Use a unified interface to switch between models.
- Choose the right model based on your use case (cost, speed, accuracy).
- You can use cloud models (OpenAI, Claude, Gemini) or local models (Ollama, Hugging Face).
- Easily integrate new providers with LangChain.
- Helps in building flexible and future-proof AI applications.
- Always handle API keys and model limits properly.

Rahul Yadav

12. Retrieval Augmented Generation (RAG) in LangChain

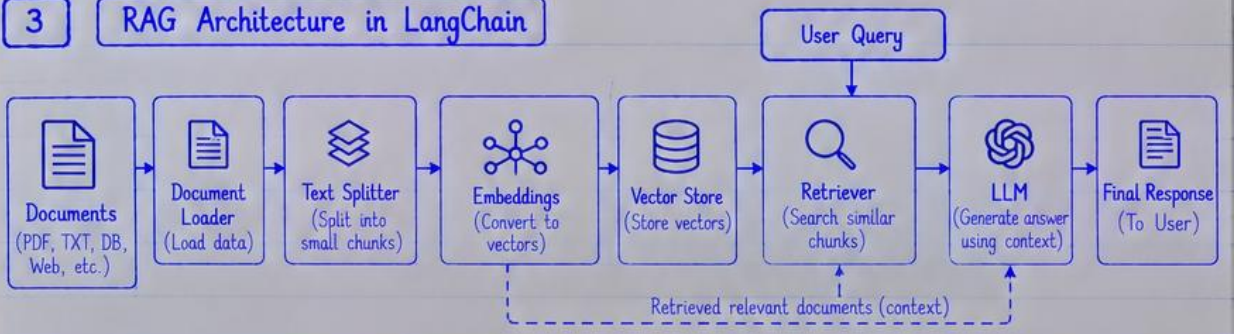
1 What is RAG ?

- RAG (Retrieval Augmented Generation) combines retrieval (search) and generation (LLM).
- It fetches relevant information from external data sources (documents, databases, etc.) and provides it to the LLM as context.
- This helps in getting accurate, up-to-date and domain-specific answers.
- Useful for chatbots, document Q&A, internal knowledge systems, etc.

2 Why Use RAG ?

- Provides accurate and factual answers.
- Uses your private data (PDFs, docs, databases).
- Reduces hallucinations in LLM responses.
- Keeps information up-to-date.
- Works with large domain-specific knowledge.
- Cost effective (you don't need to fine-tune the model).
- Useful for enterprise applications and chat with internal documents.

3 RAG Architecture in LangChain



4 Steps in RAG with LangChain

1. **Load Documents** – Load data from PDFs, text files, web pages, databases, etc.
2. **Split Documents** – Split into smaller chunks.
3. **Create Embeddings** – Convert chunks into vector embeddings.
4. **Store in Vector DB** – Store embeddings in a vector database (FAISS, Chroma, Qdrant, etc.).
5. **Retrieve Relevant Chunks** – Search similar chunks for user query.
6. **Generate Answer** – Send retrieved context to LLM and generate final response.

5 Popular Vector Stores for RAG

Vector Store	Description	Use Case
FAISS	In-memory vector store by Meta	Local development, small to medium data
Chroma	Persistent vector store, easy to use	Simple and open-source solution
Qdrant	High performance vector database	Production-ready, scalable applications
Pinecone	Managed vector database (cloud)	Scalable, enterprise use
Weaviate	Open-source vector DB with additional features	Hybrid search, production use

6 Example: Simple RAG Pipeline (PDF)

```
# Install: pip install langchain langchain-community faiss-cpu pypdf
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI

# 1. Load PDF
loader = PyPDFLoader("company_policy.pdf")
docs = loader.load()

# 2. Split into chunks
splitter = RecursiveCharacterTextSplitter(
        chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_documents(docs)
```

7 Example (Continued): Embedding, Vector Store & QA

```
# 3. Create embeddings
embeddings = HuggingFaceEmbeddings(
        model_name="all-MiniLM-L6-v2")
# 4. Store in Vector DB (FAISS)
vectorstore = FAISS.from_documents(chunks, embeddings)
# 5. Create retriever
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
# 6. Create RAG chain
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True
)
# 7. Ask a question
query = "What is the leave policy?"
result = qa_chain.invoke({"query": query})
print(result["result"])
print(result["source_documents"])
```

8 Advanced RAG Techniques

- Use metadata filtering for better retrieval.
- Hybrid search (semantic + keyword search).
- Use re-ranking models to improve result quality.
- Chunk optimization (size, overlap).
- Use context compression to fit large data.
- Combine multiple data sources (PDF + DB + API).
- Use conversation memory for multi-turn RAG.
- Evaluate RAG with metrics (relevance, accuracy).

9 Key Takeaways

- RAG combines retrieval and generation for better results.
- LangChain provides easy tools to build RAG pipelines.
- Choose the right vector store based on your use case.
- Proper chunking and embeddings are important.
- Use metadata and filtering for accurate retrieval.
- RAG is widely used in chatbots, enterprise search, and internal tools.
- It helps in building reliable and domain-specific AI applications.

Rahul Yadav

13. Chains in LangChain

1 What are Chains ?

- Chains in LangChain are sequences of calls to LLMs, tools, or other components.
- They allow you to combine multiple steps to perform a complex task.
- Chains help create modular, reusable, and maintainable workflows.
- A chain can include prompts, models, parsers, tools, memory, and more.
- LangChain provides different types of chains for different use cases.

2 Why Use Chains ?

- Break complex tasks into smaller steps.
- Combine multiple components easily.
- Reuse and maintain code.
- Support for common patterns like QA, summarization, and multi-step reasoning.
- Integrate with tools, memory, and agents.
- Build powerful applications like chatbots, document QA, and research assistants.

3 Basic Chain Architecture



4 Simple LLM Chain

A basic chain that takes a user query, formats it using a prompt template, and sends it to the LLM.

```
from langchain_openai import ChatOpenAI
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

prompt = PromptTemplate(
    input_variables=["topic"],
    template="Explain {topic} in simple words."
)

chain = LLMChain(llm=llm, prompt=prompt)

response = chain.invoke({"topic": "Microservices"})
print(response["text"])
```

5 Sequential Chain

Runs multiple chains in sequence, where the output of one chain is the input to the next.

```
from langchain.chains import SimpleSequentialChain, LLMChain

prompt1 = PromptTemplate(
    input_variables=["topic"],
    template="Explain {topic} in simple words."
)
prompt2 = PromptTemplate(
    input_variables=["explanation"],
    template="Give 3 key takeaways from this: {explanation}"
)
chain1 = LLMChain(llm=llm, prompt=prompt1, output_key="explanation")
chain2 = LLMChain(llm=llm, prompt=prompt2)

sequential_chain = SimpleSequentialChain(
    chains=[chain1, chain2],
    verbose=True
)
response = sequential_chain.invoke({"topic": "Kubernetes"})
print(response)
```

6 Types of Chains in LangChain

Chain Type	Description	Use Case
LLMChain	Basic chain with prompt and LLM	Simple tasks, Q&A
SimpleSequentialChain	Run multiple chains in sequence	Multi-step tasks
SequentialChain	Pass multiple inputs and outputs between chains	Complex workflows
RouterChain	Route query to different chains based on condition	Domain-specific QA
TransformChain	Apply custom transformation on input/output	Pre/post-processing
ConversationalChain	Chain with memory for conversations	Chatbots
RetrievalQA (Chain)	Chain for RAG using a retriever	Document Q&A

7 RetrievalQA Chain Example

Combines a retriever with an LLM to answer questions from your documents.

```
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain.chains import RetrievalQA

# Load and split documents
loader = PyPDFLoader("docs/tech.pdf")
docs = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
split_docs = splitter.split_documents(docs)

# Create vector store
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(split_docs, embeddings)
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
# Create RetrievalQA chain
qa_chain = RetrievalQA.from_chain_type(llm=llm, retriever=retriever)
response = qa_chain.invoke({"query": "What is RAG in LangChain?"})
print(response["result"])
```

8 Custom Chain (Transform Chain)

Create a custom chain to transform input or output data using custom logic.

```
from langchain.chains import TransformChain
def transform_func(inputs):
        topic = inputs["topic"]
        return {"new_topic": topic.upper()}
chain = TransformChain(
    input_variables=["topic"],
    output_variables=["new_topic"],
    transform=transform_func
)
result = chain.invoke({"topic": "langchain"})
print(result)  # {'new_topic': 'LANGCHAIN'}
```

9 Key Takeaways

- Chains help combine multiple steps to solve complex tasks.
- Use LLMChain for simple tasks.
- Use SequentialChain for multi-step workflows.
- RetrievalQA chain is useful for document-based Q&A.
- RouterChain helps route queries to different chains.
- You can create custom chains for specific requirements.
- Chains can be combined with memory, tools, and retrievers.
- Helps build modular and scalable AI applications.

Rahul Yadav

13. LangChain with Vector Databases

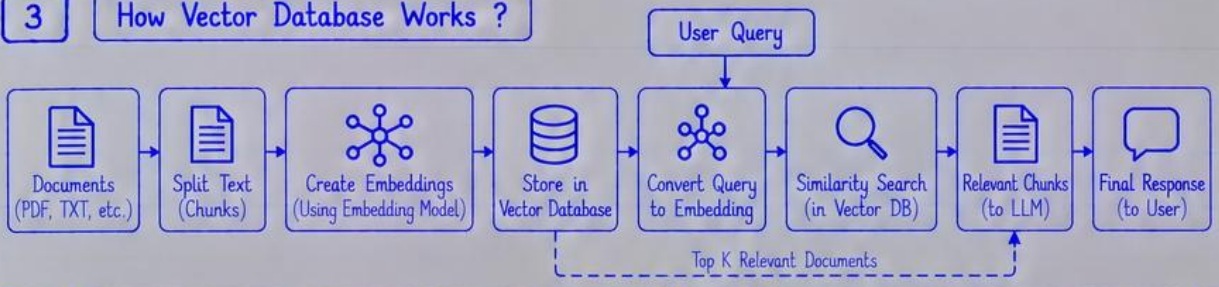
1 What are Vector Databases ?

- Vector databases store text, images, or other data as numerical vectors (embeddings).
- They enable fast similarity search to find semantically relevant data.
- Essential for RAG applications.
- Popular vector databases: FAISS, Chroma, Qdrant, Pinecone, Weaviate, Milvus, etc.
- LangChain provides easy integration with these vector databases.

2 Why Use Vector Databases ?

- Find relevant information based on meaning (semantic search).
- Works better than keyword search.
- Handles large amounts of unstructured data.
- Improves accuracy of LLM responses.
- Supports filtering (metadata-based search).
- Scalable and high-performance for enterprise use.
- Useful for document search, knowledge bases, chatbots, and RAG systems.

3 How Vector Database Works ?



4 Popular Vector Databases Comparison

Vector DB	Type	Key Features	Use Case
FAISS	Local (Library)	Fast, simple, in-memory	Local development, small to medium data
Chroma	Local/Server	Easy to use, persistent storage	Prototyping, open-source projects
Qdrant	Server/Cloud	High performance, filtering, payload	Production, scalable applications
Pinecone	Cloud	Managed service, scalable	Enterprise, production use
Weaviate	Server/Cloud	Hybrid search, metadata filtering	Production, semantic search
Milvus	Server/Cloud	High scalability, distributed	Large-scale enterprise applications

5 LangChain Vector Store Abstractions

- LangChain provides a common VectorStore interface for different databases.
- You can easily switch between databases by changing a few lines of code.
- Supports adding documents, similarity search, and metadata filtering.
- Integrates with embedding models and retrievers.
- Works seamlessly with RAG chains.

Common Methods

- add_documents() - Add documents to vector store
- similarity_search() - Search similar documents
- similarity_search_with_score() - Get score with results
- as_retriever() - Create retriever for RAG
- delete() - Delete documents
- get() - Get documents by id

6 Example 1: Using FAISS with LangChain

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import CharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS

# 1. Load and split document
loader = PyPDFLoader("data/company_policy.pdf")
docs = loader.load()
splitter = CharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(docs)

# 2. Create embeddings
embeddings = OpenAIEmbeddings()

# 3. Create FAISS vector store
vectorstore = FAISS.from_documents(chunks, embeddings)

# 4. Similarity search
results = vectorstore.similarity_search("What is the leave policy?", k=3)
for doc in results:
    print(doc.page_content)
    print("Source=", doc.metadata)
```

7 Example 2: Using Chroma with LangChain

```
from langchain.document_loaders import DirectoryLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma

# Load documents
loader = DirectoryLoader("data/")  # folder with multiple files
docs = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=500,
                                          chunk_overlap=50)
chunks = splitter.split_documents(docs)

# Create embeddings
embeddings = OpenAIEmbeddings()
# Create Chroma vector store (persictent storage)
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="chroma_db"
)
# Search
results = vectorstore.similarity_search("What is KYC process?", k=3)
for doc in results:
    print(doc.page_content)
    print("Source=", doc.metadata)
```

8 Example 3: Using Qdrant with LangChain

```
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Qdrant

embeddings = OpenAIEmbeddings()
vectorstore = Qdrant.from_documents(
    documents=chunks,
    embedding=embeddings,
    url="http://localhost:6333",   # Qdrant local server
    collection_name="company_docs"
)
```

9 Key Takeaways

- Vector databases enable semantic search for better results.
- LangChain provides a unified interface for multiple vector stores.
- Choose the right database based on scale, cost, and use case.
- Use proper chunking and embeddings for accurate search.
- Works seamlessly with RAG and agents.
- Supports metadata filtering and hybrid search.
- Essential for building scalable LLM applications.

Rahul Yadav

15. Document Loaders and Text Splitters in LangChain

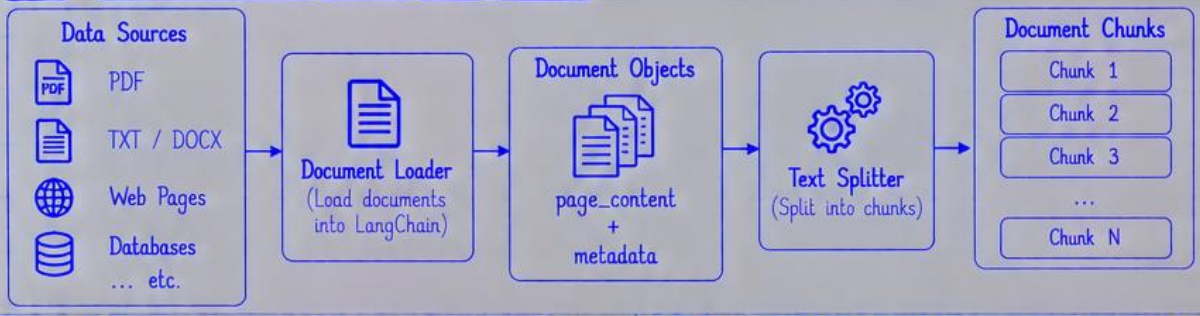
1 What are Document Loaders ?

- Document Loaders in LangChain help to load data from different sources into a standard format (Document objects).
- They support various file types like PDF, TXT, DOCX, CSV, HTML, JSON, web pages, databases, etc.
- Each loaded document contains page_content (text) and metadata (source, page number, etc.).
- Document loaders make it easy to integrate your private data with LLM applications.

2 What are Text Splitters ?

- Text Splitters divide large documents into smaller chunks.
- LLMs have token limits, so splitting helps to process large documents efficiently.
- Different splitters are available based on use case (character, token, semantic, recursive).
- Proper chunking improves retrieval accuracy and reduces irrelevant context.
- You can also set chunk size, overlap, and custom splitting rules.

3 Document Loading and Splitting Flow



4 Popular Document Loaders in LangChain

Loader	Supported Source	Use Case
PyPDFLoader	PDF files	Load PDF documents
TextLoader	Text files (.txt)	Simple text documents
Docx2txtLoader	Word documents (.docx)	Load Word files
CSVLoader	CSV files	Structured data from CSV
WebBaseLoader	Web pages (URLs)	Load data from websites
DirectoryLoader	Multiple files in a folder	Batch load documents
JSONLoader	JSON files	Load structured JSON data
UnstructuredLoader	Various formats (PDF, HTML, etc.)	Advanced loader using unstructured

5 Popular Text Splitters in LangChain

Splitter	Description	Use Case
CharacterTextSplitter	Split by character count	Simple and fast
RecursiveCharacter TextSplitter	Split recursively by characters, sentences, and other separators	Most commonly used
TokenTextSplitter	Split by tokens (e.g., using tiktoken)	When you want token-based chunks
SentenceTransformers TokenTextSplitter	Split using sentence transformers	Semantic chunking
MarkdownHeaderTextSplitter	Split based on markdown headers	Documents with headers (e.g., docs)
SemanticChunker	Split using semantic similarity	Better context preservation

6 Example 1: Load PDF and Split using RecursiveCharacterTextSplitter

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

# 1. Load PDF document
loader = PyPDFLoader("data/company_policy.pdf")
docs = loader.load()
print(f"Total pages: {len(docs)}")

# 2. Split documents into chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    separators=["\\n", "\n", "", "."]
)
splits = splitter.split_documents(docs)
print(f"Total chunks: {len(splits)}")

# 3. Check a chunk
docs[0].page_content[:200]
print(splits[0].page_content)
print(splits[0].metadata)
```

7 Example 2: Load Web Pages and Split

```
from langchain_community.document_loaders import WebBaseLoader
from langchain_text_splitters import CharacterTextSplitter

# 1. Load web pages
loader = WebBaseLoader(["https://example.com/docs"])
docs = loader.load()
print(f"Loaded {len(docs)} documents")

# 2. Split into chunks
splitter = CharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)
splits = splitter.split_documents(docs)
print(f"Total chunks: {len(splits)}")

# 3. View chunk
print(splits[0].page_content)
print(splits[0].metadata)
```

8 Useful Parameters for Text Splitters

Parameter	Description	Example
chunk_size	Size of each chunk (in characters/tokens)	1000
chunk_overlap	Overlap between chunks	200
separators	List of separators to split text	["\n\n", "\n", " "]
length_function	Function to calculate length (e.g., len or tiktoken)	len
keep_separater	Keep the separator in the chunk	True / False

9 Key Takeaways

- Document loaders help to load data from multiple sources.
- Text splitters divide large documents into smaller chunks.
- Use the right loader based on your data source.
- RecursiveCharacterTextSplitter is most commonly used.
- Proper chunk size and overlap improve retrieval accuracy.
- Split documents before creating embeddings.
- Support for PDF, Word, web pages, CSV, JSON, and more.
- Essential for building RAG applications and knowledge systems.

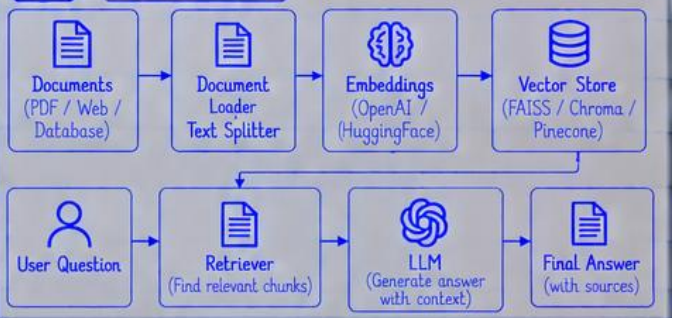
Rahul Yadav

19. Build a Complete RAG Application with LangChain

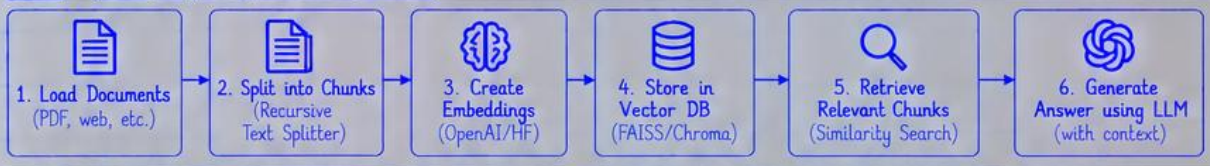
1 Overview

- A RAG (Retrieval-Augmented Generation) application combines document retrieval with LLM generation to provide accurate, up-to-date, and source-grounded answers.
- LangChain provides all the building blocks to build end-to-end RAG applications easily.
- We use document loaders, text splitters, embeddings, vector store, retriever, and LLM chain.
- RAG is widely used in chatbots, internal knowledge search, Q&A systems, and enterprise applications.

2 Architecture



3 Step-by-Step RAG Pipeline



4 Complete RAG Example (FAISS + OpenAI)

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

# 1. Load document
loader = PyPDFLoader("data/au_bank_policy.pdf")
docs = loader.load()

# 2. Split into chunks
splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
    chunk_overlap=200)
chunks = splitter.split_documents(docs)

# 3. Create embeddings and store in FAISS
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(chunks, embeddings)

# 4. Create retriever
retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

# 5. Create RAG chain
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.2)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True
)
# 6. Ask a question
response = qa_chain.invoke({"query": "What is the loan eligibility
criteria mentioned in the document?"})
print(response["result"])

# Print source documents
for doc in response["source_documents"]:
  print(doc.metadata["source"])
```

5 RAG with Conversational Memory

```
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationalRetrievalChain

# Use same vectorstore and llm from previous example
memory = ConversationBufferMemory(
    memory_key="chat_history",
    return_messages=True
)
conversational_chain = ConversationalRetrievalChain.from_llm(
    llm=llm,
    retriever=retriever,
    memory=memory,
    return_source_documents=True
)
# Chat with memory
response = conversational_chain.invoke({"question":
   "What are the key benefits?"})
print(response["answer"])
```

6 RAG with Web Pages (WebBaseLoader)

```
from langchain.document_loaders import WebBaseLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

# Load web page
loader = WebBaseLoader(["https://www.aubank.in/about-us"])
docs = loader.load()

# Split and create vector store (same as before)
splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
    chunk_overlap=200)
chunks = splitter.split_documents(docs)
vectorstore = FAISS.from_documents(chunks, embeddings)

# Use the same QA chain
qa_chain = RetrievalQA.from_chain_type(llm=llm, retriever=vectorstore.as_retriever())
response = qa_chain.invoke({"query": "What services does AU Bank provide?"})
print(response["result"])
```

7 Common Use Cases

- Chat with internal company documents (policies, manuals, FAQs).
- Customer support chatbot using product documents.
- Research assistant for technical documents.
- Summarize long documents with source references.
- Knowledge base search for enterprise applications.
- Q&A over websites, PDFs, databases, and more.

8 Useful LangChain Components

Component	Purpose
Document Loaders	Load data from PDF, web, database, etc.
Text Splitters	Split large documents into smaller chunks.
Embeddings	Convert text to vector representation.
Vector Stores	Store and search embeddings (FAISS, Chroma, Pinecone).
Retrievers	Find relevant chunks for a query.
Chains	Combine retriever + LLM for final answer.
Memory	Maintain conversation history.

9 Best Practices

- Use high-quality and clean documents.
- Choose the right chunk size (usually 500–1000).
- Use metadata and proper source attribution.
- Tune retriever parameters (k, score threshold).
- Use conversation memory for chat applications.
- Evaluate results and handle irrelevant responses.
- Combine with re-ranking for better accuracy.
- Validate outputs and show source documents.

10 Key Takeaways

- RAG combines the power of retrieval and generation.
- LangChain makes it simple to build end-to-end RAG applications.
- Use the right tools and parameters for your use case.
- RAG provides accurate, contextual, and source-grounded answers.
- It is essential for building real-world AI applications.

Rahul Yadav

18. LangChain Chains, Agents and Tools

1 What are Chains ?

- Chains in LangChain are sequences of components that work together to perform a task.
- They allow you to combine LLMs, prompts, retrievers, and other components in a pipeline.
- Chains make it easy to build complex workflows like RAG, summarization, question answering, etc.
- LangChain provides built-in chains like LLMChain, RetrievalQA, ConversationalChain, etc.

2 What are Agents ?

- Agents use an LLM to decide which tool to use, when to use it, and how to use it.
- They can interact with external tools (APIs, databases, calculators, search, etc.) to complete tasks.
- Agents can plan, take actions, observe results, and iterate until they get the final answer.
- Useful for dynamic tasks where the LLM needs to choose the right tool based on the user query.

3 Chains vs Agents vs Tools

Aspect	Chains	Agents	Tools
Purpose	Execute a fixed sequence of steps	Dynamically decide and use tools	External functions or APIs used by agents
Control Flow	Predefined	Dynamic (LLM decides)	Executed when called by agent
Use Case	RAG, summarization, Q&A pipelines	Complex tasks, multi-step reasoning	Search, database query, calculator, API calls, etc.
Flexibility	Less flexible	More flexible and autonomous	Extensible (custom tools)
Examples	LLMChain, RetrievalQA	ZeroShotAgent, OpenAIAgent	SearchTool, PythonREPLTool, Custom Tools

4 Simple Chain Example (LLMChain)

```
from langchain.llms import OpenAI
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain

# 1. Define LLM
llm = OpenAI(model="gpt-3.5-turbo", temperature=0.7)

# 2. Define prompt template
prompt = PromptTemplate(
    input_variables=["topic"],
    template="Explain {topic} in simple words.")
)
# 3. Create chain
chain = LLMChain(llm=llm, prompt=prompt)

# 4. Run chain
result = chain.invoke({"topic": "Microservices"})
print(result["text"])
```

5 Retrieval QA Chain Example

```
from langchain.chains import RetrievalQA
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.llms import OpenAI
# Load and split document
loader = PyPDFLoader("data/tech_document.pdf")
docs = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
        chunk_overlap=200)
chunks = splitter.split_documents(docs)
#  Create vector store
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(chunks, embeddings)
retriever = vectorstore.as_retriever()
#  Create RetrievalQA chain
qa_chain = RetrievalQA.from_chain_type(llm=OpenAI(),
          chain_type="stuff", retriever=retriever)
result = qa_chain.invoke({"query": "What is the main topic of this document?"})
print(result["result"])
```

6 Agents Architecture



7 Agent Example (Using Tools)

```
from langchain.agents import initialize_agent, Tool
from langchain.tools import DuckDuckGoSearchRun
from langchain.llms import OpenAI

#  Define tools
search = DuckDuckGoSearchRun()
tools = [
    Tool(
        name="Search",
        func=search.run,
        description="Useful for searching information on the internet"
    )
]
#  Create agent
llm = OpenAI(model="gpt-3.5-turbo", temperature=0.7)
agent = initialize_agent(tools, llm, agent="zero-shot-react-description",
                         verbose=True)

#  Run agent
response = agent.invoke({"input": "Find latest news about AU Small Finance Bank"})
print(response["output"])
```

8 Popular Built-in Tools

Tool	Description
DuckDuckGoSearchRun	Search information from internet
SerpAPIWrapper	Search using Google SerpAPI
WikipediaQueryRun	Search from Wikipedia
PythonREPLTool	Execute Python code
SQLDatabaseToolkit	Query a database using natural language
RequestsTool	Make HTTP API requests
Custom Tool	Create your own tool (e.g., call internal API)

9 Key Takeaways

- Chains execute a fixed sequence of steps.
- Agents dynamically decide which tool to use.
- Tools extend agent capabilities with external APIs and functions.
- Use built-in tools or create custom tools.
- Agents are useful for complex, multi-step real-world tasks.
- Combine Chains + Agents + Tools for powerful LLM applications.

Rahul Yadaw

17. Retrieval-Augmented Generation (RAG) in LangChain

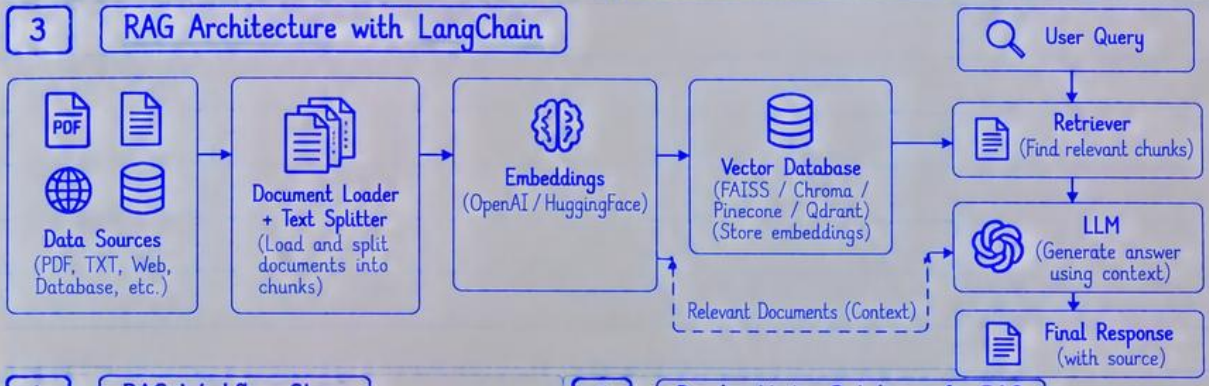
1 What is RAG ?

- Retrieval-Augmented Generation (RAG) combines retrieval (search relevant documents) with generation (LLM) to produce accurate and contextual answers.
- It helps LLMs use your private data instead of just relying on pre-trained knowledge.
- Reduces hallucinations and provides source-based answers.
- Widely used in chatbots, Q&A systems, document search, and enterprise knowledge systems.

2 Why Use RAG ?

- Provides accurate and up-to-date information from your own documents.
- Reduces hallucination by grounding answers in real data.
- Works with PDFs, websites, databases, and more.
- Supports source citations (where the information came from).
- Essential for enterprise applications and domain-specific knowledge (policies, manuals, FAQs, etc.).

3 RAG Architecture with LangChain



4 RAG Workflow Steps

1. Load documents from various sources (PDF, website, DB, etc.).
2. Split documents into smaller chunks.
3. Create embeddings and store them in a vector database.
4. When a user asks a question, retrieve relevant chunks.
5. Pass retrieved context + query to LLM.
6. Generate final answer with source information.

5 Popular Vector Databases for RAG

Vector DB	Type	Key Features	Use Case
FAISS	In-memory (by Meta)	Fast, simple, open-source	Local development, small to medium data
Chroma	Embedded	Easy to use, persistent storage	Simple projects, prototyping
Pinecone	Cloud (SaaS)	Managed, scalable, high performance	Production, large scale applications
Qdrant	Self-hosted / Cloud	High performance, filtering, payload	Production, enterprise use
Weaviate	Self-hosted / Cloud	Hybrid search, GraphQL, schema	Advanced search, enterprise applications

6 Example: Simple RAG using FAISS

```
from langchain.documentt_laoders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

# 1. Load and split document
loader = PyPDFLoader("data/company_policy.pdf")
docs = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
   chunk_overlap=200)
chunks = splitter.split_documents(docs)

# 2. Create embeddings and store in FAISS
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(chunks, embeddings)

# 3. Create retriever and QA chain
retriever = vectorstore. as_retriever(search_kwargs={"k": 3})
qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-3.5-turbo", temperature=0.2),
    retriever=retriever
)
# 4. Ask a question
response = qa_chain.invoke({"query": "What is the leave policy?"})
print(response["result"])
```

7 Example: RAG with Chroma (Persistent)

```
from langchain.document_Laaders import DirectoryLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma
from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

# 1. Load documents
loader = DirectoryLoader("data/")
docs = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
 chunk_overlap=200)
chunks = splitter.split_documents(docs)

# 2. Create embeddings and store in Chroma (persistent)
embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(
      documents=chunks,
      embedding=embeddings,
      persist_directory="chroma_db"
)
# 3. Create QA chain
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
qa_chain = RetrievalQA.from_chain_type(llm=ChatOpenAI(),
      retriever=retriever)
# 4. Query
response = qa_chain.invoke({"query": "Summarize this document."})
print(response["result"])
```

8 Key Parameters in RAG

Parameter	Description	Example
chunk_size	Size of each chunk (in characters/tokens)	1000
chunk_overlap	Overlap between chunks	200
search_kwargs(k)	Number of relevant chunks to retrieve	3
temperature	Controls randomenss in LLM response	0.0 - 0.5
model_name	LLM model to use	gpt-3.5-turbo

9 Best Practices & Key Takeaways

- Use high-quality and clean documents.
- Choose the right chunk size and overlap.
- Use a reliable vector database based on your scale and use case.
- Combine RAG with metadata filtering for better results.
- Include source citations in the final response.
- Fine-tune retriever and LLM parameters for accurate answers.
- RAG is essential for building enterprise chatbots, knowledge bases, and document Q&A systems.

Rahul Yadav

16. Embeddings and Semantic Search in LangChain

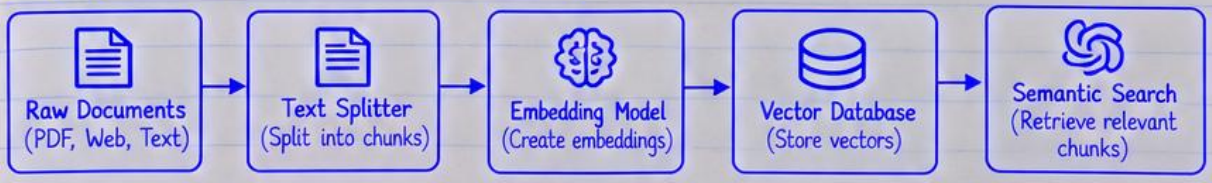
1 What are Embeddings ?

- Embeddings are numerical vector representations of text, capturing semantic meaning.
- They convert text (documents, queries) into dense vectors (e.g., 1536 dimensions) using embedding models.
- Similar content will have similar vectors, enabling semantic search, RAG, and AI applications.

2 Popular Embedding Models

- OpenAI Embeddings – text-embedding-3-small
- Cohere Embeddings – multilingual models
- Hugging Face – sentence-transformers
- Local Models – BGE, all-MiniLM, Instructor models
- Multimodal Models – for text, images, code, etc.

3 Embedding Workflow in LangChain



4 How Semantic Search Works ?

- User query is converted into an embedding vector.
- The vector is compared with stored document vectors (using similarity search).
- Top-k most relevant chunks are retrieved and used by LLM to generate accurate answers.

5 Similarity Metrics

Metric	Description	Use Case
Cosine Similarity	Measures angle between vectors	Most common
Euclidean Distance	Measures actual distance	When magnitude matters
Dot Product	Simple & efficient	Suitable for large-scale search

6 Implementing Embeddings in LangChain

```
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS

# Create embeddings
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(docs, embeddings)
```

7 Semantic Search with LangChain

```
from langchain.vectorstores import FAISS
from langchain.chains import RetrievalQA

retriever = vectorstore.as_retriever(search_kwargs={
    "k": 3
})
qa_chain = RetrievalQA.from_chain_type(llm=llm,
                retriever=retriever)
result = qa_chain.run("What is LangChain?")
print(result)
```

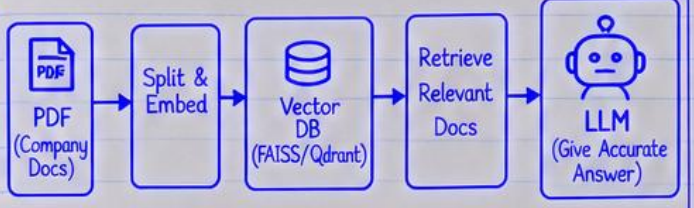
8 Advanced Techniques

- Hybrid Search – Combine keyword + vector search.
- Metadata Filtering – Filter results by source, date, tags.
- Re-ranking – Improve results using cross-encoders.
- Multi-vector Retrieval – Use multiple embeddings (e.g., title + content).
- Semantic Chunking – Split content based on meaning, not just size.

9 Use Cases of Embeddings

- RAG Applications – Retrieve relevant context for LLM.
- Semantic Search – Intelligent search in docs, products.
- Recommendation Systems – Find similar items.
- Chatbots & Assistants – Answer using your data.
- Multimodal Search – Search across text, images, code.

10 Example: RAG with Embeddings



11 Key Takeaways

- Embeddings bridge the gap between text and machine understanding.
- Semantic search helps find the most relevant information efficiently.
- LangChain makes it easy to generate embeddings, store them in vector databases, and use them for powerful RAG applications.
- Choose the right model, metric, and database based on your use case and scale.

Rahul Yadav

20. Best Practices, Challenges and Future of RAG with LangChain

1 Best Practices for RAG with LangChain

1. Use clean and high-quality documents.
2. Choose the right document loader and text splitter.
3. Use meaningful chunk size (500–1000 characters) with proper overlap (100–200).
4. Use good embedding models (OpenAI / HuggingFace).
5. Use the right vector store based on your use case.
6. Tune retriever parameters (k, score_threshold).
7. Use metadata filtering for better results.
8. Use conversational memory for chat applications.
9. Evaluate results and fine-tune prompts, chunking, and retriever settings.
10. Monitor performance, cost, and accuracy.

2 Common Challenges in RAG

Challenge	Description	Possible Solution
Poor Retrieval Results	Irrelevant or incomplete context.	Improve chunking, embeddings, metadata and retriever settings.
Hallucination	LLM generates incorrect answers.	Provide better context, use strict prompts, citation, and validation.
Large Documents	Difficult to handle long or unstructured data.	Use recursive text splitter, metadata and summarization.
Duplicate Content	Same content retrieved multiple times.	Use metadata filtering and deduplication.
Latency and Cost	High response time and API cost.	Use caching, smaller models, optimize chunk size, and use local models (Ollama).
Keeping Data Updated	Data changes frequently.	Implement re-indexing strategy and incremental updates.
Security and Privacy	Sensitive data exposure.	Use private vector DB, access control, data masking.

3 Tips to Improve RAG Accuracy

- Use domain-specific and high-quality data.
- Provide clear and specific user queries.
- Use metadata (e.g., document type, source, date).
- Combine keyword search + semantic search (hybrid).
- Use reranking for better context selection.
- Include source citations in the answer.
- Experiment with different embedding models.
- Evaluate using real user queries.

4 Evaluation Metrics for RAG

Metric	Purpose	Description
Relevance	Check if retrieved chunks are relevant.	Measures how relevant the retrieved documents are.
Answer Accuracy	Check correctness of final answer.	Compares generated answer with ground truth.
Context Precision	Quality of retrieved context.	How much of the retrieved content is actually useful.
Context Recall	Completeness of context.	How much relevant information was retrieved.
Latency	Measure response time.	Time taken to get final answer.
Cost	Monitor API usage cost.	Total tokens and cost per query.

5 Advanced Techniques

- Hybrid Search (Keyword + Semantic).
- Reranking (e.g., Cohere Rerank, OpenAI Rerank).
- Query Transformation (rewrite user query).
- Multi-Query Retrieval (generate multiple queries).
- Self-Query Retriever (structured filters).
- Context Compression (remove irrelevant parts).
- Hierarchical Indexing (summaries + detailed chunks).
- Use of Agents for dynamic tool selection.
- Caching frequent queries.
- Evaluation and continuous improvement.

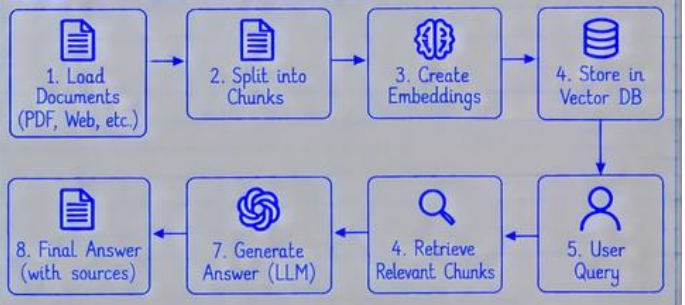
6 Future of RAG and LangChain

- Multimodal RAG (Text + Images + Audio + Video).
- Better long-context handling with advanced models.
- More powerful open-source embedding and reranking models.
- Integration with enterprise tools and real-time data.
- More advanced agents that can plan and use multiple tools.
- Improved evaluation frameworks.
- Higher accuracy with structured and unstructured data.
- Wider adoption in domain-specific applications (finance, healthcare, legal, etc.).
- Cost optimization and faster inference with local models.
- RAG will become a core building block for enterprise AI systems.

7 Complete RAG Project Structure (Example)

```
rag-app/
├── app.py              # Main application (Streamlit / Flask)
├── data/               # Store documents (PDF, TXT, etc.)
├── loaders.py          # Document loading logic
├── splitter.py         # Text splitting logic
├── embeddings.py       # Create embeddings
├── vectorstore.py      # Vector database setup (FAISS/Chroma)
├── retriever.py        # Retriever configuration
├── rag_chain.py        # RAG chain implementation
├── prompts.py          # Prompt templates
├── utils.py            # Helper functions
├── requirements.txt    # Project dependencies
└── README.md           # Project documentation
```

8 End-to-End Flow Recap



9 Key Takeaways

1. RAG combines the power of retrieval and generation.
2. LangChain makes it easy to build complete RAG applications.
3. Use the right tools, models, and parameters for your use case.
4. Focus on good data, chunking, and retrieval strategy.
5. Evaluate and continuously improve your RAG system.
6. RAG is essential for real-world, accurate, and source-grounded AI applications.

10 Final Thoughts

"RAG with LangChain is not just a technology, it's a practical way to build trustworthy, up-to-date, and domain-specific AI applications using your own data. With the right approach, it can solve real business problems and unlock the true potential of LLMs."

Rahul Yadav